Testes automatizados da API ESG

Documentação dos cenários BDD escritos em Gherkin para a API ESG Compliance. Os testes usam Java, Cucumber, Rest Assured e contratos JSON Schema.

Como os testes foram organizados

Os passos comuns enviam requisições HTTP, comparam o status recebido, validam o corpo JSON e conferem a resposta contra um schema. Cada cenário usa uma empresa de teste criada no início. Os recursos criados são excluídos ao final.

Configuração

O arquivo testes-bdd/pom.xml define Cucumber, JUnit, Rest Assured e o validador JSON Schema. O runner Java está em testes-bdd/src/test/java/com/esg/bdd/RunCucumberTest.java. O workflow está em .github/workflows/ci-cd.yml; ele inicia Oracle, compila a API e executa os cenários.

Cenários Gherkin

1. Criar, consultar, atualizar e remover empresa

O cenário envia os dados de uma empresa e valida criação (201), consulta por ID (200), atualização (200), listagem (200) e exclusão (204). As respostas com conteúdo são comparadas ao schema de empresa ou de lista.

2. Registrar emissões de carbono e conferir o total

O cenário registra uma emissão de 12,5 tCO2e, consulta as emissões da empresa e a emissão individual, verifica o total da empresa, atualiza os dados e exclui o registro. Os status esperados são 201 na criação, 200 nas consultas e atualização, e 204 na exclusão. O total esperado é 12,5.

3. Gerenciar licenças ambientais e consultar vencimentos

O cenário cria uma licença, consulta a lista da empresa, consulta a licença individual, verifica a rota de vencimentos, atualiza a licença e a exclui. São validados os status 201, 200 e 204 e os schemas correspondentes.

4. Rastrear auditoria ESG de uma empresa

O cenário cria uma auditoria ambiental com resultado conforme, consulta a lista e o registro individual, altera o resultado para parcialmente conforme e exclui a auditoria. As respostas são validadas por status e schema.

Cenários e execução

5. Rejeitar dados inválidos e acesso sem autenticação

O cenário chama a API sem token e espera 403. Depois envia uma emissão com quantidade e período inválidos e espera 400, validando o schema de erro de validação. Por fim, consulta uma empresa inexistente e espera 404.

6. Registrar usuário e autenticar credenciais válidas

O cenário registra um usuário com e-mail único e espera 201. Em seguida, faz login com as mesmas credenciais e espera 200. As respostas são conferidas pelos schemas de usuário e token.

7. Consultar saúde e informação da aplicação

O cenário consulta /actuator/health sem autenticação e espera 200. Depois consulta /actuator/info e espera 200. Os dois corpos JSON são validados por schema.

Como executar

É necessário ter Docker Desktop com Compose. Na raiz do repositório, crie e configure o arquivo .env com as senhas do Oracle e o segredo JWT. Execute:

```
Copy-Item .env.example .env
.\testes-bdd\run-tests.ps1
```

O script inicia Oracle, API e testes, encerra os serviços ao final e retorna o resultado da execução. Os relatórios locais ficam em testes-bdd/target/.

Evidência de execução

Execução no GitHub Actions: workflow 41751580002, commit 3b16205. O job de build terminou com sucesso. O job BDD terminou com sucesso e executou 7 cenários: 0 falhas, 0 erros e 0 ignorados; tempo reportado pelo Surefire: 3,870 segundos.

Execução: https://github.com/gustavogomes-dv/esg-compliance-api/actions/runs/41751580002

O XML original baixado do artefato do workflow está em output/evidence/cucumber-41751580002.xml. O GitHub Actions também conserva os relatórios Surefire como artefato relatorios-testes-bdd.